SOURCE CODE

OF

DESIGN PROJECT-1

“FUTURISTIC RADAR OBJECT DETECTION WITH CAMERA”

GROUP.NO: 26(AIML-B)

SUBMITTED

BY

S.NO	NAME	REGISTRATION.NO
1	CHITTAGARI ARUN KUMAR REDDY	2023BCSE07AED195
2	SHAIK MOHAMMED MAHABOOB BASHA	2023BCSE07AED211
3	B.C.V.GANESH	2023BCSE07AED179
4	GONAPATI VISHADHAR REDDY	2023BCSE07AED192

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8"/>
<meta name="viewport" content="width=device-width,initial-scale=1"/>
<title>Neon Nexus v7.4 — Object Recognition + Arduino Radar</title>
<style>
:root{
--neon:#00ffea; --accent:#00ffa2; --danger:#ff4d6d; --bg:#02060a; --warning:#ffcc00;
}
*{box-sizing:border-box}
body{
margin:0;padding:0;font-family:'Segoe UI',Tahoma,Geneva,Verdana,sans-serif;
color:var(--neon);
background:
radial-gradient(1200px 800px at 20% 10%, #022 0%, transparent 60%),
radial-gradient(1200px 900px at 80% 80%, #013 0%, transparent 60%),
linear-gradient(180deg, #010308 0%, #020508 100%);
text-shadow:0 0 12px var(--neon);
min-height:100vh; overflow-x:hidden;
}
header{ text-align:center; padding:20px; font-size:22px; font-weight:bold;
background: rgba(0,255,200,0.05); border-bottom:1px solid rgba(0,255,234,0.2);
box-shadow:0 0 40px rgba(0,255,200,0.2); position:relative;
}
.logo{ position:absolute; left:20px; top:50%; transform:translateY(-50%); font-size:14px;
opacity:0.8;}
.controls{ display:flex; flex-wrap:wrap; justify-content:center; gap:12px; padding:12px; }
button{ padding:10px 16px; border-radius:10px; background:linear-
gradient(180deg,#042,#011);
```

```
border:1px solid var(--accent); color:var(--accent); font-weight:700; cursor:pointer;
box-shadow:0 0 18px rgba(0,255,162,0.25); transition:all .18s ease;
}
button:hover{ transform:translateY(-2px); box-shadow:0 0 28px rgba(0,255,234,0.6); }
button.active{ background:linear-gradient(180deg,#064,#033); box-shadow:0 0 28px rgba(0,255,234,0.8); }
button:disabled{ opacity:0.5; cursor:not-allowed; }
.layout{ display:grid; grid-template-columns:1fr 1fr; gap:20px; max-width:1300px; margin:20px auto; align-items:start; }
canvas#radar{ width:580px; height:580px; max-width:100%; display:block; margin:0 auto; border-radius:50%; border:2px solid var(--accent); background: radial-gradient(circle at 50% 50%, #001b12 0%, #000 65%); box-shadow:0 0 35px var(--accent) inset;}
#video{ display:none }
#detCanvas{ width:720px; height:540px; max-width:100%; display:block; margin:0 auto; border-radius:12px; border:2px solid var(--accent); background:#020408; box-shadow:0 0 25px rgba(0,255,234,0.5); }
.status{ text-align:center; padding:12px; font-size:16px; min-height:26px; }
.status.warning{ color:var(--warning); text-shadow:0 0 12px var(--warning); }
.status.danger{ color:var(--danger); text-shadow:0 0 12px var(--danger); }
.data-panel{ max-width:1300px; margin:12px auto; display:grid; grid-template-columns:repeat(auto-fit,minmax(220px,1fr)); gap:12px; padding:12px; border-radius:10px; background:rgba(0,20,30,0.6); border:1px solid rgba(0,255,234,0.1); box-shadow:0 0 16px rgba(0,255,234,0.06);}
.data-item{ display:flex; justify-content:space-between; padding:8px; border-bottom:1px solid rgba(0,255,234,0.06); }
.data-label{ font-weight:700; opacity:0.9; }
.data-value{ color:var(--accent); font-weight:700; }
.objects-panel{ max-width:1300px; margin:12px auto; padding:12px; background:rgba(0,30,40,0.6); border-radius:12px; border:1px solid rgba(0,255,234,0.1);}
.objects-title{ text-align:center; font-size:18px; margin-bottom:10px; }
.objects-grid{ display:grid; grid-template-columns:repeat(auto-fill,minmax(200px,1fr)); gap:10px; }
```

```
.object-card{ padding:10px; border-radius:8px; background:rgba(0,0,0,0.25); border:1px solid rgba(0,255,234,0.06); }

.notification{ position:fixed; bottom:18px; right:18px; background:rgba(0,20,30,0.95); color:var(--neon); padding:12px; border-radius:8px; border:1px solid var(--accent); transform: translateX(110%); opacity:0; transition:all .35s ease; box-shadow:0 0 20px rgba(0,255,234,0.12);}

.notification.show{ transform:translateX(0); opacity:1; }

.scan-line{ position:absolute; height:2px; width:100%; background:linear-gradient(90deg, transparent, var(--neon), transparent); top:0; left:0; opacity:0; pointer-events:none; }

@media (max-width:1100px){ .layout{ grid-template-columns:1fr; } .logo{ position:static; transform:none; margin-bottom:8px; } }

</style>

</head>

<body>

<header><div class="logo">NEON NEXUS v7.4</div>Object & Animal Recognition + Arduino Radar</header>


<div class="controls">

<button id="arduinoConnect">Connect Arduino</button>

<button id="disconnect" disabled>Disconnect</button>

<button id="camToggle">Start Camera</button>

<button id="testMode">Test Mode</button>

<button id="mute">Mute</button>

<button id="fullscreen">Fullscreen</button>

</div>


<div id="status" class="status">Status: System Initialized</div>


<div class="data-panel">

<div class="data-item"><span class="data-label">Distance:</span><span id="distanceValue" class="data-value">-- cm</span></div>
```

```
<div class="data-item"><span class="data-label">Objects Detected:</span><span id="objectsValue" class="data-value">0</span></div>

<div class="data-item"><span class="data-label">Detection Confidence:</span><span id="confidenceValue" class="data-value">--</span></div>

<div class="data-item"><span class="data-label">System Uptime:</span><span id="uptimeValue" class="data-value">00:00:00</span></div>

</div>


<div class="objects-panel">

<div class="objects-title">Detected Objects & Animals</div>

<div class="objects-grid" id="objectsGrid">

<div class="object-card"><div class="object-name">No objects detected</div><div class="object-confidence">--</div><div class="object-distance">--</div></div>

</div>

</div>


<div class="layout">

<canvas id="radar" width="580" height="580" aria-label="Radar canvas"></canvas>

<div>

<video id="video" autoplay playsinline muted></video>

<canvas id="detCanvas" width="720" height="540" aligned="Detection canvas"></canvas>

</div>

</div>


<div id="notification" class="notification"></div>

<div class="scan-line" id="scanLine"></div>


<script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@4.16.0/dist/tf.min.js"></script>

<script src="https://cdn.jsdelivr.net/npm/@tensorflow-models/coco-ssd"></script>
```

```
<script>
// UI elements
const statusEl = document.getElementById("status");
const btnCam = document.getElementById("camToggle");
const btnArduino = document.getElementById("arduinoConnect");
const btnDisconnect = document.getElementById("disconnect");
const btnMute = document.getElementById("mute");
const btnFullscreen = document.getElementById("fullscreen");
const btnTestMode = document.getElementById("testMode");
const notificationEl = document.getElementById("notification");
const objectsGridEl = document.getElementById("objectsGrid");
const distanceValueEl = document.getElementById("distanceValue");
const objectsValueEl = document.getElementById("objectsValue");
const confidenceValueEl = document.getElementById("confidenceValue");
const uptimeValueEl = document.getElementById("uptimeValue");
const scanLineEl = document.getElementById("scanLine");

// Canvas contexts
const rCanvas = document.getElementById("radar");
const rCtx = rCanvas.getContext("2d");
const dCanvas = document.getElementById("detCanvas");
const dCtx = dCanvas.getContext("2d");
const video = document.getElementById("video");

// App state
let objectModel = null;
let isDetecting = false; // Is camera running?
let detectedObjects = 0;
let detectionConfidence = 0;
```

```
let muteEnabled = false;
let testModeEnabled = false;

// Arduino + Radar state
let port = null;
let reader = null;
let sensorDistance = null;      // real sensor distance in cm (from Arduino)
const MAX_SENSOR_RANGE = 400;   // cm (tunable)

// Beep system
let audioCtx = null, beepTimer = null;

// *** STATE FOR VOICE FEEDBACK LOGIC ***
// Distance Stability Check (3 seconds constant distance)
const STABILITY_DURATION = 3000; // 3 seconds
const STABILITY_TOLERANCE = 5; // cm
let currentObservedDistance = null; // Used by the updateSensorData loop
let observationStartTime = 0;
let lastAnnouncedDistance = null;

// Object Name Cooldown (Immediate announcement, 1.5s cooldown)
const SPEAK_OBJECT_COOLDOWN = 1500;
const lastSpokenObjectName = {};

// Single distance value currently displayed and used for stability
let currentDashboardDistance = null;
// ************************************

// Utility functions
```

```
const isArduinoConnected = () => port !== null || testModeEnabled;

const setStatus = (t, type = "") => {
statusEl.textContent = "Status: " + t;
statusEl.className = "status";
if (type) statusEl.classList.add(type);
};

const showNotification = (message, type = "info") => {
notificationEl.textContent = message;
notificationEl.className = "notification";
if (type === "warning") notificationEl.classList.add("warning");
if (type === "danger") notificationEl.classList.add("danger");
notificationEl.classList.add("show");
setTimeout(() => notificationEl.classList.remove("show"), 3800);
};

// --- Beep system ---
function playBeep(distance) {
if (!audioCtx) audioCtx = new (window.AudioContext || window.webkitAudioContext)();
const osc = audioCtx.createOscillator();
const gain = audioCtx.createGain();

let freq = 200 + Math.max(0, (1000 - distance * 2.5));
freq = Math.max(200, Math.min(1200, freq));
osc.frequency.value = freq;

gain.gain.value = 0.3;
osc.connect(gain).connect(audioCtx.destination);
```

```
osc.start();
osc.stop(audioCtx.currentTime + 0.12);
}

function updateBeep(distance) {
if (beepTimer) clearInterval(beepTimer);
if (!distance) return;
let interval = Math.min(2000, Math.max(150, distance * 8));
beepTimer = setInterval(() => playBeep(distance), interval);
}

function speak(text) {
if (muteEnabled) return;
if (!text) return;
// Cancel any previous, pending speech before starting a new one
window.speechSynthesis.cancel();
const u = new SpeechSynthesisUtterance(text);
u.rate = 0.95;
u.volume = 1;
u.pitch = 1;
window.speechSynthesis.speak(u);
}

const objectDatabase = {
'dog': { type: 'animal', icon: '🐕', category: 'Mammal' },
'cat': { type: 'animal', icon: '🐈', category: 'Mammal' },
'bird': { type: 'animal', icon: '🐦', category: 'Bird' },
'horse': { type: 'animal', icon: '🐎', category: 'Mammal' },
'sheep': { type: 'animal', icon: '🐑', category: 'Mammal' },
```

'cow': { type: 'animal', icon: '🐄', category: 'Mammal' },

'elephant': { type: 'animal', icon: '🐘', category: 'Mammal' },

'bear': { type: 'animal', icon: '🐻', category: 'Mammal' },

'zebra': { type: 'animal', icon: '🦓', category: 'Mammal' },

'giraffe': { type: 'animal', icon: '🦒', category: 'Mammal' },

'lion': { type: 'animal', icon: '🦁', category: 'Mammal' },

'tiger': { type: 'animal', icon: '🐅', category: 'Mammal' },

'book': { type: 'object', icon: '📚', category: 'Stationery' },

'cell phone': { type: 'object', icon: '📱', category: 'Electronics' },

'laptop': { type: 'object', icon: '💻', category: 'Electronics' },

'mouse': { type: 'object', icon: '🖱️', category: 'Electronics' },

'keyboard': { type: 'object', icon: '⌨️', category: 'Electronics' },

'cup': { type: 'object', icon: '☕', category: 'Container' },

'bottle': { type: 'object', icon: '🧴', category: 'Container' },

'chair': { type: 'object', icon: '🪑', category: 'Furniture' },

'dining table': { type: 'object', icon: '🍽️', category: 'Furniture' },

'tv': { type: 'object', icon: '📺', category: 'Electronics' },

'remote': { type: 'object', icon: '📟', category: 'Electronics' },

'microwave': { type: 'object', icon: '🍲', category: 'Appliance' },

'oven': { type: 'object', icon: '🔥', category: 'Appliance' },

'toaster': { type: 'object', icon: '🍞', category: 'Appliance' },

'sink': { type: 'object', icon: '🚰', category: 'Fixture' },

'refrigerator': { type: 'object', icon: '❄️', category: 'Appliance' },

'clock': { type: 'object', icon: '⏰', category: 'Decor' },

'vase': { type: 'object', icon: '🏺', category: 'Decor' },

'scissors': { type: 'object', icon: '✂️', category: 'Tool' },

```
'teddy bear': { type: 'object', icon: '🧸', category: 'Toy' },
'hair drier': { type: 'object', icon: '💨', category: 'Appliance' },
'toothbrush': { type: 'object', icon: '🪥', category: 'Hygiene' }
};

/********** Camera + Model setup **********/
async function setupCamera() {
try {
setStatus("Starting camera...");
const stream = await navigator.mediaDevices.getUserMedia({ video: true });
video.srcObject = stream;
await video.play();
btnCam.classList.add("active");
btnCam.textContent = "Stop Camera";
setStatus("Loading AI model...");
dCtx.clearRect(0, 0, dCanvas.width, dCanvas.height);

objectModel = await cocoSsd.load();

setStatus("AI model loaded, Camera ON", "warning");
showNotification("Object recognition model loaded", "success");
isDetecting = true;
detectObjects();
} catch (err) {
console.error("Camera error:", err);
setStatus("Camera error: " + err, "danger");
showNotification("Camera error. Check permissions.", "danger");
}
}
```

```
function stopCamera() {
if (video.srcObject) {
video.srcObject.getTracks().forEach(t => t.stop());
video.srcObject = null;
}
window.speechSynthesis.cancel();
btnCam.classList.remove("active");
btnCam.textContent = "Start Camera";
isDetecting = false;
dCtx.clearRect(0, 0, dCanvas.width, dCanvas.height);
setStatus("Camera stopped. Sensor mode active.");

// Clear object-related dashboard info
objectsValueEl.textContent = 0;
confidenceValueEl.textContent = "--";
updateObjectsPanel(null);
}

// Estimate distance from bounding box height if Arduino absent
function estimateDistance(bbox) {
const refHeight = dCanvas.height;
const objectHeight = Math.max(4, bbox[3]); // avoid division by zero
const distance = Math.round((refHeight / objectHeight) * 50);
return Math.max(20, Math.min(distance, MAX_SENSOR_RANGE)); // clamp to [20, MAX_SENSOR_RANGE]
}

// The main detection loop (runs only when camera is ON)
```

```
async function detectObjects() {
if (!isDetecting || !objectModel) return;

dCtx.clearRect(0, 0, dCanvas.width, dCanvas.height);
dCtx.drawImage(video, 0, 0, dCanvas.width, dCanvas.height);
const sensorActive = isArduinoConnected();
const now = performance.now();

try {
const predictions = await objectModel.detect(video);
detectedObjects = predictions.length;
objectsValueEl.textContent = detectedObjects;

let distanceSourceText = '-- cm';
let cameraDistance = null;

if (predictions.length > 0) {
// Use the distance and class of the highest confidence prediction
const topPrediction = predictions[0];
const cls = topPrediction.class;
cameraDistance = estimateDistance(topPrediction.bbox);

// --- 1. IMMEDIATE OBJECT NAME ANNOUNCEMENT (Camera ON Only) ---
const lastTimeSpoken = lastSpokenObjectName[cls] || 0;
if (now - lastTimeSpoken > SPEAK_OBJECT_COOLDOWN) {
lastSpokenObjectName[cls] = now;
// Speak the object name immediately
speak(`${cls} detected.`);
}
```

```
// ------------------------------------------------------------

if (sensorActive && sensorDistance) {
// Priority 1: Use sensor data
currentDashboardDistance = sensorDistance;
distanceSourceText = `${sensorDistance} cm`;
} else {
// Priority 2: Use camera estimate
currentDashboardDistance = cameraDistance;
distanceSourceText = `${cameraDistance} cm (Estimated)`;
}

distanceValueEl.textContent = distanceSourceText;

// Draw boxes for all predictions
predictions.forEach(prediction => {
const [x, y, width, height] = prediction.bbox;
const objectInfo = objectDatabase[prediction.class] || { icon: '❓', type: 'unknown',
category: 'Unknown' };

dCtx.strokeStyle = "#00ffa2";
dCtx.lineWidth = 3;
dCtx.strokeRect(x, y, width, height);

dCtx.fillStyle = "rgba(0,20,30,0.8)";
dCtx.fillRect(x, y - 28, 260, 26);
dCtx.font = "16px Arial";
dCtx.fillStyle = "#00ffa2";
dCtx.fillText(`${objectInfo.icon} ${prediction.class} (${Math.round(prediction.score *
100)}%)`, x + 8, y - 7);
```

```
});

const totalConfidence = predictions.reduce((s, p) => s + p.score, 0);
detectionConfidence = (totalConfidence / predictions.length * 100).toFixed(1);
confidenceValueEl.textContent = `${detectionConfidence}%`;
setStatus("Detecting...");

} else {
// No camera predictions, only sensor if active
confidenceValueEl.textContent = "--";

if (sensorActive && sensorDistance) {
// Camera OFF, but sensor has data
currentDashboardDistance = sensorDistance;
} else {
// Nothing detected by either camera or sensor
currentDashboardDistance = null;
}
}

// warning for many objects
if (detectedObjects >= 5) {
setStatus(`Warning: ${detectedObjects} objects detected`, "warning");
showNotification(`High object count: ${detectedObjects} detected`, "warning");
}

updateObjectsPanel(predictions);

} catch (err) {
```

```
console.error("Detection error:", err);
}

// Continue detection loop
requestAnimationFrame(detectObjects);
}

// *** Permanent loop for Sensor/Dashboard/Voice updates (runs regardless of camera state) ***
function updateSensorData() {
const sensorActive = isArduinoConnected();
const now = performance.now();
let distanceToUse = null;
let distanceSource = "";

if (isDetecting) {
// Camera is ON: use the distance determined by detectObjects
distanceToUse = currentDashboardDistance;
if (sensorActive && sensorDistance) {
distanceSource = "sensor calculated";
} else if (currentDashboardDistance !== null) {
distanceSource = "camera estimated";
}
// The dashboard value (distanceValueEl) is already set in detectObjects

} else {
// Camera is OFF: only use sensor data for distance
if (sensorActive && sensorDistance) {
distanceToUse = sensorDistance;
```

```
distanceSource = "sensor calculated";
distanceValueEl.textContent = `${distanceToUse} cm`;
} else {
distanceToUse = null;
distanceValueEl.textContent = '-- cm';
}
}

// Update beep based on the distance available
if (distanceToUse) {
updateBeep(distanceToUse);
} else {
updateBeep(null);
}

// *** DISTANCE STABILITY CHECK (3s delay) ***
if (distanceToUse !== null) {
// Check for stability and reset observation time if unstable
if (currentObservedDistance === null || Math.abs(distanceToUse -
currentObservedDistance) > STABILITY_TOLERANCE) {
currentObservedDistance = distanceToUse;
observationStartTime = now;
}

// Check if distance has been stable for 3 seconds AND is a new distance
if (now - observationStartTime >= STABILITY_DURATION &&
lastAnnouncedDistance !== currentObservedDistance) {
lastAnnouncedDistance = currentObservedDistance;
```

```
let speech = `Present distance ${Math.round(currentObservedDistance)} centimeters, ${distanceSource}.`;
speak(speech);
}
} else {
// No object or valid sensor reading, clear stability state
currentObservedDistance = null;
observationStartTime = 0;
lastAnnouncedDistance = null;
// Only cancel if camera is also off, to allow object name speech if camera is on
if(!isDetecting) window.speechSynthesis.cancel();
}
}
// Run this function frequently to ensure distance stability is always monitored
setInterval(updateSensorData, 500);


// Update objects panel
function updateObjectsPanel(predictions) {
objectsGridEl.innerHTML = "";
if (!predictions || predictions.length === 0) {
objectsGridEl.innerHTML = `<div class="object-card"><div class="object-name">No objects detected</div><div class="object-confidence">--</div><div class="object-distance">--</div></div>`;
return;
}
predictions.sort((a, b) => b.score - a.score);
predictions.forEach(prediction => {
const objectInfo = objectDatabase[prediction.class] || { icon: '❓', type: 'unknown', category: 'Unknown' };
```

```
const sensorActive = isArduinoConnected();
const cameraDistance = estimateDistance(prediction.bbox);
const distance = sensorActive && sensorDistance ? sensorDistance : cameraDistance;
const distanceSource = sensorActive && sensorDistance ? 'Sensor' : 'Estimated';

const card = document.createElement("div");
card.className = "object-card";
card.innerHTML = `
<div class="object-name">${objectInfo.icon} ${prediction.class}</div>
<div class="object-confidence">Confidence: ${Math.round(prediction.score * 100)}%</div>
<div class="object-distance">Distance: ${distance} cm (${distanceSource})</div>
<div class="object-type">Type: ${objectInfo.type} | ${objectInfo.category}</div>
`;
objectsGridEl.appendChild(card);
});
}

/********** Arduino Web Serial integration **********/
async function connectArduino() {
if (!("serial" in navigator)) {
showNotification("Web Serial API not supported in this browser. Use Chrome/Edge.", "danger");
return;
}
const BAUDRATE = 9600;
try {
setStatus("Requesting Arduino port...");
port = await navigator.serial.requestPort();
await port.open({ baudRate: BAUDRATE });
```

```
showNotification(`Arduino connected (Baud: ${BAUDRATE})`, "success");
setStatus("Arduino connected", "warning");
btnArduino.disabled = true;
btnDisconnect.disabled = false;

// Say sensor connected when Arduino connects
speak("Sensor connected. Using sensor calculated distance.");

// reader loop
readSensorLoop().catch(e => console.error("readSensorLoop err", e));
} catch (err) {
console.error("Arduino connection error:", err);
// Improved error message for user
showNotification(`Arduino connection failed. Check cable and ensure correct Baud Rate (${BAUDRATE}).`, "danger");
setStatus("Arduino connection failed", "danger");
}
}

async function disconnectArduino() {
try {
if (reader) { await reader.cancel(); reader.releaseLock(); reader = null; }
if (port) { await port.close(); port = null; }
sensorDistance = null;
currentDashboardDistance = null;
distanceValueEl.textContent = '-- cm';
btnArduino.disabled = false;
btnDisconnect.disabled = true;
if (beepTimer) clearInterval(beepTimer);
```

```
window.speechSynthesis.cancel();
showNotification("Arduino disconnected", "warning");
setStatus("Arduino disconnected");

// Reset distance stability state
currentObservedDistance = null;
observationStartTime = 0;
lastAnnouncedDistance = null;
} catch (err) {
console.error("Arduino disconnect error", err);
}
}

// robust read loop that accumulates newline-terminated numbers
async function readSensorLoop() {
const textDecoder = new TextDecoder();
const readerStream = port.readable.getReader();
let buffer = "";
reader = readerStream;
try {
while (true) {
const { value, done } = await readerStream.read();
if (done) break;
if (value) {
buffer += textDecoder.decode(value, { stream: true });
// process lines
let nl;
while ((nl = buffer.indexOf("\n")) >= 0) {
const line = buffer.slice(0, nl).trim();
```

```
buffer = buffer.slice(nl + 1);
if (line.length > 0) {
// try parse integer
const num = parseInt(line.replace(/[^\d\-]/g, ""), 10);
if (!isNaN(num)) {
// treat valid ranges only
if (num > 0 && num <= 2000) {
sensorDistance = num;
} else {
// if sensor sends out-of-range, treat as no object
sensorDistance = null;
}
}
}
}
}
}
} catch (err) {
console.error("readSensorLoop error:", err);
} finally {
try { readerStream.releaseLock(); } catch { }
}
}

/********** Radar drawing **********/
function drawRadar() {
rCtx.clearRect(0, 0, rCanvas.width, rCanvas.height);
const cx = rCanvas.width / 2, cy = rCanvas.height / 2;
const radius = rCanvas.width / 2 - 12;
```

```
// grid circles
rCtx.strokeStyle = "rgba(0,255,234,0.12)";
rCtx.lineWidth = 1.2;
for (let i = 1; i <= 4; i++) {
rCtx.beginPath();
rCtx.arc(cx, cy, (radius / 4) * i, 0, Math.PI * 2);
rCtx.stroke();
}

// radial lines
rCtx.strokeStyle = "rgba(0,255,234,0.06)";
for (let a = 0; a < Math.PI * 2; a += Math.PI / 6) {
rCtx.beginPath();
rCtx.moveTo(cx, cy);
rCtx.lineTo(cx + radius * Math.cos(a), cy + radius * Math.sin(a));
rCtx.stroke();
}

// *** FIX APPLIED: Use PI/2 (90 degrees) to fix the dot to the South (straight down) ***
// 90 degrees (Math.PI / 2) is the South direction in standard Canvas coordinates.
const fixedSouthAngle = Math.PI / 2;

// If sensorDistance available and valid, plot dot at fixed South angle
if (sensorDistance && sensorDistance > 0 && sensorDistance <=
MAX_SENSOR_RANGE) {
const scale = radius / MAX_SENSOR_RANGE;
const dist = Math.min(sensorDistance, MAX_SENSOR_RANGE);
```

```
// The X and Y coordinates are calculated using the fixed angle and the dynamic distance (dist * scale)
const x = cx + (dist * scale) * Math.cos(fixedSouthAngle);
const y = cy + (dist * scale) * Math.sin(fixedSouthAngle);

rCtx.beginPath();
rCtx.fillStyle = "#ff4d6d";
rCtx.arc(x, y, 7, 0, Math.PI * 2);
rCtx.fill();

// numeric overlay
rCtx.fillStyle = "rgba(0,0,0,0.6)";
rCtx.fillRect(10, 10, 140, 36);
rCtx.fillStyle = "#00ffa2";
rCtx.font = "16px Arial";
rCtx.fillText(`Sensor: ${sensorDistance} cm`, 18, 34);
} else {
// show numeric fallback or '--'
rCtx.fillStyle = "rgba(0,0,0,0.6)";
rCtx.fillRect(10, 10, 140, 36);
rCtx.fillStyle = "#00ffa2";
rCtx.font = "16px Arial";
rCtx.fillText(`Sensor: --`, 18, 34);
}

requestAnimationFrame(drawRadar);
}
drawRadar();
```

```
/********** Uptime counter **********/
const startTime = Date.now();
function updateUptime() {
const elapsed = Date.now() - startTime;
const h = Math.floor(elapsed / 3600000);
const m = Math.floor((elapsed % 3600000) / 60000);
const s = Math.floor((elapsed % 60000) / 1000);
uptimeValueEl.textContent = `${String(h).padStart(2, '0')}:${String(m).padStart(2, '0')}:${String(s).padStart(2, '0')}`;
}
setInterval(updateUptime, 1000);

/********** Test Mode **********/
function toggleTestMode() {
testModeEnabled = !testModeEnabled;
btnTestMode.textContent = testModeEnabled ? "Exit Test Mode" : "Test Mode";
btnTestMode.classList.toggle("active", testModeEnabled);

if (testModeEnabled) {
// Simulate sensor data
const testInterval = setInterval(() => {
if (!testModeEnabled) {
clearInterval(testInterval);
return;
}
// Generate random distance between 20 and 200 cm
sensorDistance = Math.floor(Math.random() * 180) + 20;
}, 1000);
```

```
showNotification("Test mode activated", "warning");
setStatus("Test mode: Simulating sensor data", "warning");

// Say sensor connected when Test Mode starts (as it simulates the sensor)
speak("Test sensor activated. Using sensor calculated distance.");

} else {
sensorDistance = null;
if (beepTimer) clearInterval(beepTimer);
showNotification("Test mode deactivated", "info");
setStatus("System ready");

// Reset distance stability state
currentObservedDistance = null;
observationStartTime = 0;
lastAnnouncedDistance = null;
}
}

/********** Controls/events **********/
btnCam.addEventListener("click", () => {
if (video.srcObject) stopCamera();
else setupCamera();
});

btnArduino.addEventListener("click", () => {
connectArduino();
});
```

```
btnDisconnect.addEventListener("click", () => {
disconnectArduino();
});

btnMute.addEventListener("click", () => {
muteEnabled = !muteEnabled;
btnMute.textContent = muteEnabled ? "Unmute" : "Mute";
if (muteEnabled) window.speechSynthesis.cancel();
});

btnFullscreen.addEventListener("click", () => {
if (!document.fullscreenElement) document.documentElement.requestFullscreen().catch(() => { });
else document.exitFullscreen().catch(() => { });
});

btnTestMode.addEventListener("click", toggleTestMode);

// friendly initial messages
setStatus("System initialized");
showNotification("Neon Nexus ready — press Start Camera & Connect Arduino", "info");
</script>
</body>
</html>
```